

GUÍA DE INTEGRACIÓN PARA DESARROLLADORES

Payment_Card_Service

Sesiones de pago con tarjeta de débito y crédito

Servicio	Payment_Card_Service
Base URL	https://payment-service-card-production.up.railway.app/api/payments/
Audiencia	Equipos de desarrollo de comercios integradores
Última actualización	20 de agosto de 2026

Documento técnico de referencia — uso interno y para comercios integradores

1. Introducción

Payment_Card_Service es un servicio que permite generar una sesión de pago con tarjeta de débito o crédito. El flujo entrega una URL de checkout asociada a un token opaco de un solo uso, donde el cliente final ingresa los datos de su tarjeta para completar el pago.

El procesamiento interno de la tarjeta (autorización, captura, etc.) es manejado completamente por el servicio y no requiere integración directa por parte del comercio.

2. Información general

Nombre del servicio	Payment_Card_Service
Base URL	https://payment-service-card-production.up.railway.app/api/payments/
Formato	JSON

3. Flujo general de integración

- El comercio solicita una sesión de pago (POST /session) indicando monto, moneda y código de referencia.
- El servicio responde con un token opaco de un solo uso y una URL de checkout.
- El comercio redirige al cliente final a esa URL, donde ingresa los datos de la tarjeta.
- Cuando el pago se resuelve, Payment_Card_Service notifica al comercio mediante los 2 servicios webhook que el comercio debe implementar (ver 4.3).
- El comercio consulta el estado de sus pagos (POST /search, ver 4.4) usando la misma API key de comercio (Authorization: Bearer) que usó para crear la sesión en 4.1 — no hace falta un paso de autenticación previo.

4. Endpoints

4.1 Crear sesión de pago

Método	POST
Endpoint	/session
Autenticación	Authorization: Bearer {merchant_notification_api_key} (header, no en el body)

Request

```
{
  "code": "123456789",
  "amount": 10,
  "currency": "BOB",
  "language": "esp",
  "descripcion": "Pago de servicio",
```

```

    "redirect": false,
    "urlToRedirect": "https://comercio.example.com/gracias"
  }

```

Campo	Tipo	Descripción
code	string	Código o referencia de la transacción/orden.
amount	number	Monto a cobrar.
currency	string	Moneda (ej. BOB).
language	string	Opcional. "esp" o "eng" (cualquier otro valor cae en español). Define el idioma del checkout, el comprobante y la pantalla de error.
descripcion	string	Opcional. Texto libre que se muestra en el recuadro "Servicio" del checkout. Si se omite, se usa "Pago de servicio".
redirect	boolean	Opcional, default false. Si es true, la pantalla final (éxito o error) redirige automáticamente a urlToRedirect después de 10 segundos.
urlToRedirect	string (URL)	Obligatorio si redirect=true. URL absoluta http(s). Si redirect=false, solo se usa como destino del botón "Volver al inicio"/"Back to home".

Ejemplo curl

```

curl -X POST "https://payment-service-card-production.up.railway.app/api/payments/session" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer {merchant_notification_api_key}" \
-d '{
  "code": "123456789",
  "amount": 10,
  "currency": "BOB",
  "language": "esp",
  "descripcion": "Pago de servicio",
  "redirect": false,
  "urlToRedirect": "https://comercio.example.com/gracias"
}'

```

Response

```

{
  "token": "9d44aa7837689c0c630892ff9b6a778e302a3bfb...",
  "url": "https://payment-service-card-production.up.railway.app/payments/9d44aa7837689c0c630892ff9b6a778e302a3bfb...",
  "expires_in": 900
}

```

Campo	Tipo	Descripción
token	string	Token opaco de un solo uso.
url	string	URL de checkout a la que debe redirigirse al cliente.
expires_in	number	Tiempo de expiración en segundos (900 = 15 min).

Nota: los valores de token, password y JWT mostrados en este documento están truncados/enmascarados como ejemplo. No representan credenciales reales.

4.2 Página de checkout

Método	GET
Endpoint	/payments/{token}
Tipo	Interfaz web (no devuelve JSON)

Al visitar esta URL (obtenida en la respuesta de /session) se carga la interfaz de checkout donde el cliente final completa el pago.

La pantalla muestra:

- Referencia de la transacción (code).
- Monto y moneda a pagar.
- Marca de tarjeta detectada (Visa, Mastercard, etc.).
- Formulario: número de tarjeta, nombre del titular, mes/año de expiración y CVV.

Nota: el procesamiento de la tarjeta ocurre de forma interna en el servicio; los endpoints que reciben los datos de la tarjeta no son públicos y no requieren integración por parte del comercio.

A continuación se muestran las dos pantallas principales de este flujo.

Pantalla de pago

Vista de resumen (izquierda) con la referencia, el monto y el servicio a pagar, junto al formulario de datos de la tarjeta (derecha) donde el cliente ingresa número, titular, fecha de expiración, CVV y sus datos de contacto/facturación.

SECURE CHECKOUT

Pago de servicio

Realiza tus pagos de forma segura.

REFERENCIA#123456789

123456789

MONTO

Visa

SERVICIO

Pago de servicio

TOTAL A PAGAR10,00 BOB

CHECKOUT

Pago seguro

Datos del pago

VISA

4000 0000 0000 1000

TITULAR

NOMBRE Y APELLIDO

CVV ---

EXPIRA

12/28

Número de tarjeta

4000000000001000

Nombre del titular

Roberto Jimenez

Mes

Año

12

2028

CVV

Correo electrónico

...

cliente@example.com

Nombre

Apellido

Roberto

Jimenez

Dirección

Calle Herber 123

Ciudad

País

La Paz

BO

Tus datos se usan sólo para procesar este pago.

Pagar ahora

Nota: esta captura corresponde a una versión anterior de la interfaz. El diseño actual usa una paleta oscura/naranja unificada con el comprobante y ya no muestra un recuadro de "Monto" separado (el monto sigue visible en "Total a pagar").
Pantalla de checkout: resumen del pago y formulario de tarjeta

Comprobante de pago

Una vez que el pago se procesa exitosamente, se muestra un comprobante con el número de referencia, el estado ("Pagado"), el método de pago usado, el ID de transacción, la fecha y el monto total pagado.

Copyright Sentinel 2026

COMPROBANTE DE PAGO

123456789

PAGADO

RESUMEN DEL PAGO

MÉTODO
Tarjeta **** 1000

TRANSACCIÓN
7871897039346297703814

FECHA
20/08/2026 01:35

CONTACTO
cliente@example.com

TOTAL PAGADO

BOB 10,00

Gracias por su compra. Conserve este comprobante como respaldo de su transacción.

Imprimir comprobante Volver al inicio

Nota: esta captura corresponde a una versión anterior de la interfaz. El diseño actual comparte la misma paleta oscura/naranja del checkout y ya no usa tonos verdes; además puede mostrar el aviso de redirección automática descrito arriba.

Comprobante de pago mostrado al finalizar la transacción

Idioma y redirección automática

El parámetro `language` ("esp" o "eng") determina el idioma de las tres pantallas del flujo: el formulario de checkout, el comprobante de pago y la pantalla de error. Si no se envía o el valor no es reconocido, se usa español por defecto.

Si `redirect` es false (o no se envía), `urlToRedirect` solo se usa como destino del botón "Volver al inicio"/"Back to home" de la pantalla final. En ese caso también se muestra, debajo de los botones, una nota indicando que hay que presionar ese botón para finalizar.

Si `redirect` es true, la pantalla final además muestra un aviso de cuenta regresiva ("Redirigiendo en 10 segundos...") y redirige automáticamente a `urlToRedirect` a los 10 segundos — tanto si el pago fue exitoso como si falló. En ese caso se ocultan el botón "Imprimir comprobante" y el texto de agradecimiento de la pantalla de éxito, ya que el cliente será redirigido de todas formas.

4.3 Pautas para el comercio: servicios de notificación de pago (webhooks)

Cuando el pago se resuelve, `Payment_Card_Service` notifica al comercio llamando a dos endpoints que el comercio debe implementar en su propio sistema. A continuación se describen las pautas de diseño que el comercio debe seguir.

4.3.1 Idea general

En vez de usar una API key fija para el webhook (que si se filtra sirve indefinidamente), el comercio emite un token opaco de un solo uso por cada transacción. `Payment_Card_Service` debe presentar ese token exacto para poder actualizar el estado del pago, de forma que cada notificación sea válida una única vez y únicamente para esa transacción.

- El comercio crea la sesión de pago y, antes de recibir la notificación, expone un endpoint (Servicio A) donde Payment_Card_Service solicita un token opaco de notificación.
- Cuando el pago se resuelve, Payment_Card_Service llama al segundo endpoint del comercio (Servicio B) presentando ese token para actualizar el estado de la transacción.
- El comercio valida el token (existe, no expiró, no fue usado) y lo invalida tras procesarlo, de forma que no pueda reutilizarse.

4.3.2 Servicio A (comercio): emitir token opaco de notificación

Método	POST
Endpoint (comercio)	/webhooks/payments/token
Quién llama	Payment_Card_Service
Autenticación	API key o credenciales fijas del comercio

Request (Payment_Card_Service → Comercio)

```
{
  "bookCode": "123456789",
  "status": "Pagado"
}
```

Ejemplo curl (Payment_Card_Service llamando al comercio)

```
curl -X POST "https://comercio.example.com/api/webhooks/payments/token" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer {api_key_del_comercio}" \
-d '{
  "bookCode": "123456789",
  "status": "Pagado"
}'
```

Response (Comercio → Payment_Card_Service)

```
{
  "success": true,
  "notification_token": "b7e2f1...",
  "expires_in": 300
}
```

Reglas para el comercio:

- Generar un token aleatorio criptográficamente seguro (mínimo 32 bytes, ej. random_bytes + bin2hex, o UUID v4 largo).

- Guardar en base de datos: el token (o su hash), el bookCode asociado, used_at = null y expires_at (ej. +5 minutos).
- No guardar el token en texto plano si se puede evitar — igual que un password, guardar hash(token) y comparar hashes.

4.3.3 Servicio B (comercio): recibir notificación y actualizar el pago

Método	PATCH o POST
Endpoint (comercio)	/webhooks/payments/{notification_token}
Quién llama	Payment_Card_Service
Autenticación	El propio token opaco en la URL o header

Request

```
{
  "bookCode": "123456789",
  "status": "Pagado",
  "paymentDate": "2026-08-19 23:57:38",
  "amount": "10.00",
  "currency": "BOB"
}
```

Ejemplo curl (Payment_Card_Service llamando al comercio)

```
curl -X PATCH
"https://comercio.example.com/api/webhooks/payments/b7e2f1a9c3d84e6f1b2a7c9e0d4f5a8b3c6e1d9f2a7b4c8e0d1f6a3b9c2e5d7f" \
-H "Content-Type: application/json" \
-d '{
  "bookCode": "123456789",
  "status": "Pagado",
  "paymentDate": "2026-08-19 23:57:38",
  "amount": "10.00",
  "currency": "BOB"
}'
```

Response esperada

```
{ "success": true }
```

Reglas de validación que el comercio debe aplicar, en este orden:

- 1. Buscar el token por su hash — calcular hash(notification_token recibido) y buscarlo en la tabla de tokens. Si no existe, responder 401/404 sin dar detalles (no revelar si el token existe o expiró, para evitar enumeración).
- 2. Verificar que no haya sido usado — si used_at ya tiene valor, rechazar la petición (409 Conflict). Evita que la misma notificación se procese dos veces por reintentos de red.
- 3. Verificar que no haya expirado — comparar la hora actual contra expires_at. Si expiró, rechazar (410 Gone) y loggear el intento.
- 4. Verificar que el bookCode coincida — debe ser el mismo con el que se generó el token en el Servicio A. Si no coincide, rechazar, para evitar que un token válido se use para actualizar una transacción distinta.
- 5. Actualizar la transacción y marcar el token como usado — todo dentro de una transacción de base de datos (DB transaction) para que sea atómico: actualizar status/paymentDate del pago y marcar used_at = now() en el mismo commit. Evita condiciones de carrera si llegan dos peticiones casi simultáneas.
- 6. Responder 200 con success true — solo después de que el commit fue exitoso. Si algo falla, hacer rollback y responder error para que Payment_Card_Service pueda reintentar (con un token nuevo, ya que este queda inválido).

4.3.4 Esquema de tabla sugerido (MySQL, lado comercio)

```
CREATE TABLE webhook_notification_tokens (
  id          BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  token_hash  CHAR(64) NOT NULL UNIQUE,      -- sha256(token)
  book_code   VARCHAR(64) NOT NULL,
  used_at     DATETIME NULL,
  expires_at  DATETIME NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_book_code (book_code)
);
```

Consideraciones extra de seguridad recomendadas para el comercio:

- HTTPS obligatorio en ambos endpoints — el token viaja en la URL o body, y sin TLS es interceptable.
- Timeout corto: 5 minutos es razonable para expires_in (es solo el tiempo entre "voy a notificar" y "notifico").
- Idempotencia: si Payment_Card_Service reintenta por timeout de red, el comercio debe responder de forma predecible (409) sin romper nada.
- Logging: registrar cada intento fallido (token inválido, expirado, ya usado) para detectar abuso o bugs de integración.
- Opcional pero recomendable: firmar el body con HMAC usando un secreto compartido, como capa extra además del token.

4.4 Consultar estado de pago(s)

Método	POST
Endpoint	/search

Autenticación	Authorization: Bearer {merchant_notification_api_key} (misma API key que 4.1, sin paso previo)
----------------------	--

Headers

```
Authorization: Bearer {merchant_notification_api_key}
```

Request

```
{
  "from_date": "2026-08-20",
  "to_date": "2026-08-21",
  "bookCode": ""
}
```

Campo	Tipo	Descripción
from_date	string	Fecha inicial del rango de búsqueda (opcional).
to_date	string	Fecha final del rango de búsqueda (opcional).
bookCode	string	Código de la transacción a buscar (opcional).

Nota: los criterios de búsqueda son excluyentes: se debe buscar por rango de fechas (from_date/to_date) o por bookCode, pero no por ambos al mismo tiempo.

Ejemplo curl

```
curl -X POST "https://payment-service-card-production.up.railway.app/api/payments/search" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer {merchant_notification_api_key}" \
-d '{
  "from_date": "2026-08-20",
  "to_date": "2026-08-21",
  "bookCode": ""
}'
```

Response

```
{
```

```

"success": true,
"user": {
  "id": 1,
  "username": "testuser",
  "email": "test@example.com"
},
"count": 1,
"payments": [
  {
    "bookCode": "123456789",
    "amount": "0",
    "currency": "BOB",
    "description": "Pago online",
    "status": "Pagado",
    "creationDate": "2026-08-19 23:57:38",
    "paymentDate": "2026-08-19 23:57:38"
  }
],
"data": [
  {
    "bookCode": "123456789",
    "amount": "0",
    "currency": "BOB",
    "description": "Pago online",
    "status": "Pagado",
    "creationDate": "2026-08-19 23:57:38",
    "paymentDate": "2026-08-19 23:57:38"
  }
]
}

```

Campo	Tipo	Descripción
success	boolean	Indica si la búsqueda fue exitosa.
user	object	Datos del usuario autenticado.
count	number	Cantidad de resultados encontrados.
payments	array	Listado de pagos encontrados, con la misma estructura que data.
data	array	Duplicado de payments, con idéntica estructura y contenido.

Nota:
payments y data contienen exactamente la misma información en la respuesta.

Campo	Tipo	Descripción
Se documentan ambos campos porque el servicio los devuelve así, pero para integrar basta con leer uno de los dos (se recomienda payments, por ser el nombre más descriptivo).		

5. Códigos de error y manejo de excepciones

Esta tabla consolida los mensajes de error tal como están implementados actualmente en el servicio. Son los mensajes reales que puede recibir un comercio o el cliente final, agrupados por área.

5.1 Autenticación y sesión

HTTP	Mensaje	Dónde aparece
401	Credenciales inválidas.	Autenticación (/session, /search)
422	code, amount y currency son obligatorios.	Emisión de sesión (/session)
422	Este código ya tiene un pago registrado. Genere un nuevo código para un nuevo intento.	Emisión de sesión (/session) — el code ya se usó
422	The payment session token is invalid.	Sesión de pago (checkout)
422	The payment session is not available anymore.	Sesión de pago (checkout)
422	The payment session has expired.	Sesión de pago (checkout)
422	The payment session is not available for payment.	Sesión de pago (checkout)

Nota: todas las respuestas de error de esta API (401, 409, 422, etc.) se

HTTP	Mensaje	Dónde aparece
devuelven siempre en formato JSON, sin importar el header Accept que envíe el cliente.		

5.2 Validación de tarjeta y flujo de pago

HTTP	Mensaje	Dónde aparece
409	Esta reserva ya fue pagada.	Procesamiento de pago
422	Esta reserva fue cancelada y no puede procesarse.	Procesamiento de pago
422	El número de tarjeta no es válido. (decline_code: invalid_number)	Procesamiento de pago
422	La tarjeta está vencida. (decline_code: expired_card)	Procesamiento de pago
422	No se pudo obtener referenceId o accessToken desde authentication_setup.	Procesamiento de pago — inicialización 3DS
422	Datos de pago no encontrados.	Procesamiento de pago
422	authenticationTransactionId no encontrado.	Procesamiento de pago — validación 3DS

Nota: cuando el error es de tarjeta (invalid_number / expired_card), la respuesta 422 incluye además un campo decline_code en el JSON con ese valor, útil para diferenciar el motivo exacto del rechazo.

5.3 Errores de gateway / 3DS

HTTP	Mensaje	Dónde aparece
—	Error al Procesar el Pago	Procesamiento de pago (pantalla de error al cliente)
—	Error en Autenticación 3DS	Procesamiento de pago (pantalla de error al cliente)
—	La autenticación 3DS no fue exitosa.	Procesamiento de pago
—	La sesión de pago expiró o fue alterada.	Procesamiento de pago

Nota: estos mensajes no siempre vienen con un código HTTP de error propio del servicio: el proveedor de pagos (gateway) rechaza la transacción y el servicio traduce ese resultado a una pantalla HTML de error para el cliente final, en vez de a un JSON.

Nota: estos textos se muestran en inglés cuando la sesión se creó con language: "eng" (ver 4.1); en cualquier otro caso se muestran en español,

HTTP	Mensaje	Dónde aparece
como en los ejemplos de esta tabla.		

5.4 Errores de persistencia / infraestructura

HTTP	Mensaje	Dónde aparece
500	Payment DB persistence failed.	Procesamiento de pago — fallo al guardar el resultado en base de datos
—	Commerce notification failed.	Servicio de notificación al comercio (webhooks, ver 4.3)

Nota: ambos se registran como log de warning/error internamente. En particular, un fallo de notificación al comercio no bloquea ni afecta la respuesta que recibe el cliente final durante el pago — por eso se recomienda que el comercio use /search (4.4) como verificación adicional del estado de sus transacciones.

5.5 Observación importante

El servicio no cuenta con una enumeración central de "códigos de error" como catálogo formal. Los errores están distribuidos entre validaciones de controlador, excepciones de servicios, respuestas HTML renderizadas, y logs de warning/error.

Por eso, esta tabla es una consolidación de los mensajes ya existentes en la implementación actual, no un catálogo oficial predefinido. Si el mensaje exacto cambia en una futura versión del servicio, esta sección debe actualizarse.